Portfolio Assignment: Implement Dependency Injection in the Parking System Application

ICT 4315: Object Oriented Methods and Programming II

for

Master of Science

Information Technology

Kalika Browder

University of Denver College of Professional Studies

June 7, 2026

Faculty: Nathan Braun, MBA
Director: Cathie Wilson, M.S.
Dean: Michael J. McGuire, MLS

The DU Parking System project was built entirely throughout both of my Object Oriented Programming courses, but developed thoroughly in this one, ICT-4315. . The application models the University's parking infrastructure: customers register with the parking office, receive permits for their vehicles, and are charged fees when they enter or exit the University's parking lots. Each weekly assignment layered new complexity onto the system - all the way from basic object-oriented modeling, through design patterns (Strategy, Factory, Decorator, Observer, Command), to a full multithreaded JSON client-server architecture in Assignment 9 of this specific class. It was a tough system to crack, with each layer adding new complexity constantly and therefore making me a better system designer.

Assignment 10 represents the capstone of this journey, and the apex as to what this system has become.. Its primary technical goal is to integrate Dependency Injection (DI) using the Google Guice framework. Dependency Injection is a design principle that decouples the creation of objects from their use: instead of a class calling new SomeDependency() internally, the dependency is *injected* from outside, usually by a framework. This produces code that is easier to test, easier to change, and easier to understand because every dependency relationship is declared explicitly.

The requirements for this assignment were the following: (1) collect and integrate all classes from prior assignments, incorporating any feedback; (2) refactor the application to use Guice for managing dependencies; (3) update unit tests to reflect the changes and demonstrate DI's testability benefits; and (4) produce this report with UML diagrams, lessons learned, reflections, and future enhancements.

Design

My system is organized into four conceptual layers. I made sure that each holds clear and direct responsibilities.

Package	**Responsibility**
parking	Core domain: Customer, Car, ParkingLot, ParkingOffice, ParkingService, Server
parking.charges.strategy	Strategy pattern: pricing algorithms (VehicleType, TimeOfDay)
parking.charges.factory	Factory pattern: selects the right strategy for a lot type
parking.charges.decorator	Decorator pattern: composable pricing modifiers (compact, weekend, prime-time, special day)
parking.inject	Guice module, injected server, and application entry point

Table 1: Parking Structure

Dependency Injection (Before &After)

The most illustrative change between Assignment 9 and Assignment 10 is how ParkingService and ParkingOffice are created and shared. Before DI, the original Server class hard-coded object creation:

```
//Assignment 9 — manual wiring inside Server constructor:

ParkingOffice office = new ParkingOffice("DU Parking", addr);

this.service = new ParkingService(office);
```

This is problematic because the Server class controls both the *creation* and the *use* of its dependencies. Unit tests cannot substitute a different ParkingOffice without subclassing Server or using reflection. With Guice, the approach became:

```
/ Assignment 10 — ParkingModule.java:

@Provides @Singleton

public ParkingOffice provideParkingOffice() {

   return new ParkingOffice(officeName, officeAddress);
```

```
}
```

```
/ InjectedServer.java:
@Inject
public InjectedServer(ParkingService service) {
   this.service = service; // Guice supplies this
}
```

Now ParkingModule is the single authoritative place where all objects are constructed. Tests create their own injector with a test-configured module and get a clean, isolated object graph every time. The application entry point becomes:

```
//ParkingSystemApp.java:
Injector injector = Guice.createInjector(new ParkingModule("DU Parking", officeAddress));
InjectedServer server = injector.getInstance(InjectedServer.class);
server.start();
```

Guice walks the entire dependency graph automatically in the sense that InjectedServer needs a ParkingService, which needs a ParkingOffice, which is declared as a singleton in the

module. All of this is resolved at injector creation time, with clear error messages if any binding is missing.

Key Class Relationships (Dependency Diagram)

ParkingSystemApp

└── creates Guice Injector (ParkingModule)

├── @Singleton ParkingOffice

├── @Singleton TransactionManager

├── @Singleton PermitManager

└── @Singleton ParkingService(@Inject ← ParkingOffice)

└── @Inject → InjectedServer(@Inject ← ParkingService)

└── submits ClientHandler to thread pool

└── calls ParkingService.performCommand()

└── Command.execute()

└── ParkingOffice.register()

Each arrow represents a dependency that Guice resolves — no class below ParkingModule contains a new call for another major domain object.

Server Request Lifecycle (Sequence Diagram)

Step	**Actor**	**Actor**
1	ParkingSystemApp.main()	Creates Guice.createInjector(new ParkingModule(...))
2	Guice Injector	Constructs ParkingOffice (singleton), ParkingService (singleton)
3	Guice Injector	Injects ParkingService into InjectedServer via @Inject
4	InjectedServer	Binds port 5001, starts accepting connections
5	ServerClient	Sends JSON: {"commandName":"CUSTOMER","params":{...}}
6	InjectedServer	Accepts socket, submits new ClientHandler(socket, service) to thread pool
7	ClientHandler	Reads JSON line, calls ParkingRequest.fromJson()
8	ClientHandler	Calls service.performCommand("CUSTOMER", params)
9	ParkingService	Looks up

		RegisterCustomerCommand, calls execute(params)
10	RegisterCustomerCommand	office.register(name, address, phone) → returns Customer
11	ClientHandler	Wraps result in ParkingResponse(200, message), sends JSON back
12	ServerClient	Receives and prints response

Table 2: Server Request Lifecycle, Guice Edition

Design Patterns Summary

My DU Parking System also employs six distinct design patterns, each addressing a different dimension of the problem altogether:

Pattern	**Where Used**	**Problem Solved**
Dependency Injection (Guice)	ParkingModule, InjectedServer, ParkingService	Decouples object creation from use; enables testability
Strategy	ParkingChargeStrategy hierarchy	Swappable pricing algorithms without changing ParkingLot
Factory	ParkingChargeStrategyFactory	Selects the correct strategy for each ScanType
Decorator	ParkingChargeCalculator	Composable pricing modifiers

	hierarchy	(compact, weekend, prime-time, special day)
Observer	ParkingLot (Subject), ParkingObserver (Observer)	Decouples charge recording from lot scanning logic
Command	Command interface, RegisterCustomerCommand, RegisterCarCommand	Encapsulates requests as objects; new commands require no changes to existing code

Table 3: Design Patterns Summary

Testing With Dependency Injection

One of the clearest demonstrations of DI's value is in ParkingModuleTest.java. Because dependencies are injected rather than hard-coded, each test creates its own isolated injector:

```
@BeforeEach
public void setUp() {
    Address testAddress = new Address("2199 S. University Blvd.", "Denver", "CO",
"80208");
    injector = Guice.createInjector(new ParkingModule("Test Office", testAddress));
}
```

```
@Test
public void parkingServiceAndOfficeShareSameInstance() {
   ParkingOffice  office  = injector.getInstance(ParkingOffice.class);
   ParkingService service = injector.getInstance(ParkingService.class);
   assertSame(office, service.getOffice()); // Guice singleton guarantee
}
```

This test would be impossible to write cleanly without DI because truly there would be no way to guarantee that the ParkingOfficeretrieved directly was the same instance the ParkingService was using, because both would have been constructed in separate new calls buried in class constructors.

Lessons Learned

The most significant new concept in this assignment for me was Dependency Injection as a framework-driven practice rather than a theoretical principle. Before this course, I was familiar with passing dependencies through constructors, but I had not used a dedicated DI container like Guice.. This assignment really threw me for a loop. Learning Guice revealed to me, however, how much invisible wiring a DI framework handles: the Guice Injector walks the entire dependency graph, respects scope annotations like @Singleton, and surfaces missing

bindings as clear startup errors rather than runtime null pointer exceptions discovered mid-request.

Something I learned heavily was understanding the distinction between the @Provides and @Inject annotations. @Provides lives in a Module and tells Guice *how* to build something. @Inject lives on a constructor and tells Guice *where* to deliver a dependency. Together they form a complete contract that replaces scattered new calls throughout the codebase. The module becomes a living map of the entire application's dependency graph, or a form of executable architecture documentation.

I also deepened my understanding of singleton scope in a server context. Because multiple client threads concurrently use the same ParkingOffice and ParkingService, those objects must be singletons. This is for both correctness (shared customer list) and for efficiency (no redundant instantiation per request). Guice's @Singleton annotation makes this intent explicit and enforced, whereas previously the singleton behavior was implicit and easy to accidentally break by creating a second instance in a test setup.

Working across nine prior assignments also reinforced the value of incremental refactoring to me. Adding DI to the parking system required touching only three existing files (ParkingService, Server, Main) and creating four new ones (ParkingModule, InjectedServer, ParkingSystemApp, ParkingModuleTest). The rest of the codebase was entirely unchanged. This was only possible because prior assignments correctly separated concerns. For example, if the Command or Strategy classes had known about the server, DI adoption would have been far messier.

Reflections on Improvements

Looking back across the ten assignments, the area I would approach most differently is interface extraction. The current codebase uses concrete classes almost everywhere: ParkingOffice, TransactionManager, and PermitManager are all concrete with no corresponding interface. DI frameworks work most powerfully when dependencies are expressed as interfaces. So for example, a CustomerRepository interface with an InMemoryCustomerRepository for production and a MockCustomerRepository for tests. Without interfaces, tests must inject real instances even when a lightweight stub would suffice, and swapping an implementation (say, replacing in-memory storage with a database) requires touching every class that holds a reference.

A related challenge for me was thread safety. The Assignment 9 solution wrapped mutable lists in Collections.synchronizedList, which is a pragmatic fix but not ideal for a high-throughput production system. I would instead use ConcurrentHashMap and CopyOnWriteArrayList throughout, or adopt a proper persistence layer where thread safety is handled by the storage engine's transaction semantics. Had I started with a repository interface, swapping the storage backend would have been a one-line change in ParkingModule.

I also wish I had introduced richer error types earlier. The system communicates errors as strings prefixed with "ERROR:", which is workable but fragile. Callers must do string comparisons to detect failure modes. A small hierarchy of domain exceptions

(CustomerNotFoundException, DuplicateLicenseException, UnknownCommandException) would make error handling explicit and type-safe, and would map cleanly to HTTP status codes in the server response without the current string-parsing logic in ClientHandler.

Finally, I underestimated how valuable early API design would have been. The JSON protocol (hand-rolled, no library) evolved organically. If I had defined a proper request/response schema upfront, even just a simple interface contract, the serialization code would have been more consistent and the client would have been easier to write. This is a lesson about the cost of deferred design: decisions made late are harder to change than decisions made early, even simple ones like field naming conventions.

Future Enhancements

Given additional time with the assignment, instead of just ten weeks, several enhancements would meaningfully improve the system for me, as the designer:

Persistent Storage (JPA/SQL): Currently all data lives in memory and is lost on server restart. A JPA-backed repository injected via Guice would make the system production-ready without changing any business logic. The beauty of DI here is that ParkingModule would simply swap new InMemoryCustomerRepository() for new JpaCustomerRepository(entityManager). Nothing else in the codebase changes.

REST API (JAX-RS or Spring Boot): The hand-rolled JSON-over-raw-socket protocol works but is non-standard. Replacing it with a proper REST API would enable integration with web

frontends, mobile apps, and third-party tools like Postman. Endpoints would map naturally: POST /customers, POST /cars, POST /lots/{id}/entry.

Authentication and Authorization: Any client can currently register customers or cars with no credentials. Adding JWT-based authentication, itself injectable via Guice, would gate operations to authorized users. A parking attendant would have different permissions than an administrator, modeled as roles checked in a SecurityFilter.

Billing Module: The original spec mentions monthly billing but it remains unimplemented. A BillingService (injected, scheduled via a Guice ScheduledExecutorService binding) that aggregates TransactionManager data and generates invoices would complete the core business requirement. This is the feature most directly visible to end users.

Guice-Assisted Injection for ParkingLot: ParkingLot still creates its ParkingChargeStrategy via a static factory call inside its constructor. Refactoring to @AssistedInject would allow Guice to inject the strategy while lot-specific values (lot ID, name, fees) remain caller-supplied parameters. This would make pricing strategy substitution testable at the individual lot level.

Web Dashboard: A simple HTML dashboard showing live lot occupancy, recent transactions, and per-customer charges would make the system immediately useful to parking office staff. Built as a thin REST client consuming the parking API, it would require no changes to the backend.

Conclusion

The DU Parking System project provided a rare opportunity to build a non-trivial software system from scratch, week by week, experiencing firsthand how design decisions compound over time. For me, this meant a lot, as I have now worked with code that I never thought I would before. Starting from simple POJOs in Week 1 and arriving at a Guice-injected, multithreaded, pattern-rich application in Week 10 made abstract concepts (Strategy, Observer, Dependency Injection) concrete and memorable in a way that no isolated exercise could.

The integration of Google Guice in Assignment 10 was a fitting capstone because it touched every layer of the system. To adopt DI correctly, the code had to be well-structured: classes needed clear constructors, single responsibilities, and stable dependency relationships. Where those conditions were met, adopting Guice was seamless (adding @Inject to ParkingService and writing ParkingModule took less than an hour. Where they were not) the scattered static factory calls, the missing interfaces, the concrete-class-everywhere approach, Guice exposed the technical debt immediately and honestly.

Something I really admired and were a couple key achievements come to mind were e: a fully functional client-server parking application; six design patterns applied purposefully to real problems; comprehensive JUnit 5 test coverage including new tests that verify Guice wiring and singleton guarantees; and a codebase that is demonstrably easier to test and extend than its pre-DI predecessor. The lessons learned here (about dependency management, concurrent shared state, incremental design, and the long-term cost of deferred decisions) will inform every software project for me going forward. I don’t think there’s even an instance where I

don’t remember this project because it tested me in so many ways and made me round out my skills like never before.